

Tensors Store Model Data and Parameters

A PyTorch tensor is a shaped numerical data structure used for inputs, targets, model parameters, and intermediate results.



CORE CONCEPT

One Neuron Is a Weighted Sum and Activation

A neuron calculates: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ then applies an activation function:

x_i : input feature

w_i : learned weight

b : learned bias

z : pre-activation weighted sum

g : activation function

a : neuron output

[RUN IT AND INSPECT THE RESULT](#)

Layers Transform Batches with Matrix Multiplication

```
layer = torch.nn.Linear(3, 5)
x = torch.randn(4, 3)
z = layer(x)
print(z.shape)
```

OUTPUT

```
torch.Size([4, 5])
```

Four observations each receive five transformed features.

REASONING SEQUENCE

Activations Let Networks Learn Nonlinear Patterns

Frame

An activation function transforms each pre-activation value.

1

Transform

$\text{ReLU}(z) = \max(0, z)$

2

Validate

Without nonlinear activations, several linear layers collapse into one linear transformation.

3

Explain

Nonlinearity lets the network represent curved and piecewise relationships.

4

Input values [-2, 0.5, 3] ReLU: $\max(0, -2)=0$ $\max(0, 0.5)=0.5$ $\max(0, 3)=3$ Output [0, 0.5, 3].

CORE CONCEPT

A Forward Pass Produces Predictions

A forward pass sends input tensors through layers and activations to produce predictions. Example network: $X \rightarrow \text{Linear}(3,5) \rightarrow \text{ReLU} \rightarrow \text{Linear}(5,1) \rightarrow \text{output}$

A forward pass sends input tensors through layers and activations to produce predictions.

$X \rightarrow \text{Linear}(3,5) \rightarrow \text{ReLU} \rightarrow \text{Linear}(5,1) \rightarrow \text{output}$

The output of one stage becomes the input to the next.

Shapes must align at every boundary.

The final output may be a regression prediction, a binary logit, or one logit per class.

After first linear layer: (4,5) After ReLU: (4,5) After output layer: (4,1)

RUN IT AND INSPECT THE RESULT

The Loss Must Match the Task and Output

```
loss_fn = torch.nn.MSELoss()  
prediction = torch.tensor([2.0, 4.0])  
target = torch.tensor([3.0, 5.0])  
print(loss_fn(prediction, target))
```

OUTPUT

tensor(1.)

Residuals [1,1] Squared residuals [1,1] MSE $(1+1)/2 = 1$

CORE CONCEPT

Backpropagation Computes Gradients

PyTorch autograd records tensor operations in a computational graph. During backpropagation, the chain rule moves backward from loss and calculates: $\partial \text{loss} / \partial \text{parameter}$

PyTorch autograd records tensor operations in a computational graph.

During backpropagation, the chain rule moves backward from loss and calculates:

$\partial \text{loss} / \partial \text{parameter}$

for every trainable parameter.

Calling `loss.backward()` computes gradients.

It does not update the weights.

REASONING SEQUENCE

The Optimizer Applies the Gradients

Frame

`optimizer.zero_grad()`
clears old gradients;

1

Transform

`prediction = model(x)`
runs the forward
pass;

2

Validate

`loss.backward()`
computes new
gradients;

3

Explain

`optimizer.step()`
updates parameters.

4

Current weight 1.0, gradient -12, learning rate 0.1: $w_{new} = 1.0 - 0.1(-12) = 2.2$ For the example on Slide 7, this moves the prediction from 2 toward target 5.

[RUN IT AND INSPECT THE RESULT](#)

Batches, Epochs, and Learning Rate Control Training

```
import math

updates = math.ceil(240 / 32) * 5
print(updates)
```

OUTPUT

40

For 5 epochs: $8 \times 5 = 40$ updates.

QUALITY GATE

Training and Evaluation Modes Behave Differently



`model.eval()`



`enter torch.inferencemode()`



move batch to the model device



calculate predictions and metrics

QUALITY GATE

A Complete Training Loop Produces Evidence



input and target shapes;



forward output shape;



loss value;



gradient calculation;

GUIDED LAB

Guided Lab: Trace One Batch End to End

01

**input and target
tensor shapes**

02

**device and
dtype**

03

**every layer's input
and output shape**

04

**weighted sum and
activation for one
simple neuron**